
PROBABILISTIC FORECASTING VIA AUTOREGRESSIVE FLOW MATCHING

Anonymous authors
Anonymous submission

ABSTRACT

We introduce FLOWTIME, a generative model for probabilistic forecasting of multivariate timeseries data. Given historical measurements and optional future covariates, we cast forecasting as sampling from a learned conditional distribution over future trajectories. We decompose the joint distribution of future observations into a sequence of conditional densities, each modeled by a shared flow that transforms a simple base distribution into the next observation distribution, conditioned on observed covariates. We realize this decomposition with the flow matching (FM) framework, which delivers scalable and simulation-free learning of these transformations. Coupling this factorization with the FM objective lets FLOWTIME inherit the core strengths of autoregressive models—strong extrapolation, a compact model size, and well-calibrated uncertainty estimates—while simultaneously capturing the complex, multi-modal conditional distributions that characterize modern transport-based generative models. Across multiple dynamical systems and real-world forecasting tasks, FLOWTIME consistently delivers accurate, well-calibrated forecasts, establishing autoregressive flow matching as an effective approach to probabilistic timeseries forecasting.

1 INTRODUCTION

Probabilistic timeseries forecasting is a core problem in modern machine learning: the aim is to extrapolate how system dynamics evolve into the future given observational data. It sits at the center of a wide range of scientific, industrial and societal disciplines (Lim & Zohren, 2021; Dama & Sinoquet, 2021; Ye et al., 2024).

Deep generative models have become the dominant tool for this problem (Karl et al., 2016; Rasul et al., 2020; Desai et al., 2021). In this setting, forecasting is framed as sampling from a future probability density conditioned on the past. Diffusion and score-based generative models (Sohl-Dickstein et al., 2015; Ho et al., 2020; Song et al., 2020) have driven state-of-the-art performance across multiple forecasting benchmarks (Rasul et al., 2021; Tashiro et al., 2021; Kollovieh et al., 2023; Meijer & Chen, 2024). Their accuracy, however, comes at a steep price: diffusion models incur high computational costs during both training and inference.

Flow Matching (FM) (Liu et al., 2022; Albergo et al., 2023; Lipman et al., 2022) is a powerful paradigm for generative modeling that generalizes and subsumes diffusion models while opening up far more flexible design choices. Where diffusion models rely on iterative stochastic denoising steps over a discretized trajectory, FM learns deterministic probability paths that transform arbitrary base distributions directly into the target distribution through continuous normalizing flows. By designing these probability paths optimally, FM eliminates the handcrafted noise schedules and lengthy ancestral sampling chains inherent to diffusion, yielding markedly more efficient training and sampling.

FM has recently been applied to timeseries modeling and probabilistic forecasting (Tamir et al., 2024; Hu et al., 2024; Kollovieh et al., 2024), delivering strong empirical results and improved computational efficiency over diffusion models. These approaches, however, share a fundamental limitation: they learn the conditional distribution of an entire fixed-horizon future window conditioned on a fixed-horizon context window. This design buys fast training and sampling, but it comes at a real cost—the resulting models extrapolate poorly beyond the training distribution and produce miscalibrated uncertainty estimates. It also creates a needlessly hard optimization problem, forcing

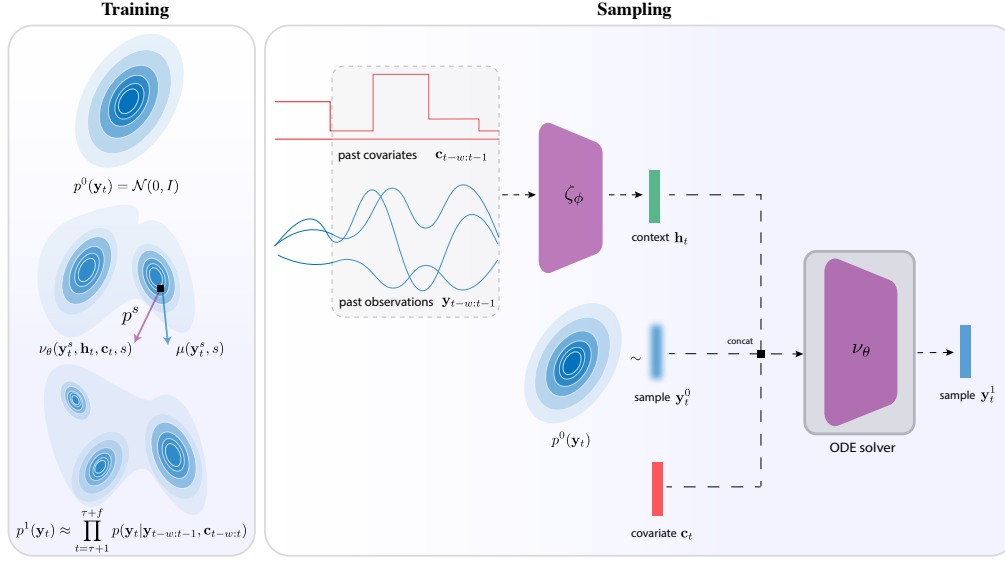


Figure 1: Overview of the FLOWTIME training and sampling process. During training, a probability path $(p^s)_{0 \leq s \leq 1}$ connects a base distribution p^0 to the target distribution p^1 , generated by a velocity field μ . We train by regressing μ with a neural network ν_θ that takes a sample from the probability path $\mathbf{y}_t^s$ at flow step s , a context vector $\mathbf{h}_t$ encoding past observations and covariates, and the current covariate c_t . At sampling time, we draw from the base distribution p^0 and integrate the trained velocity field with an ODE solver until $s = 1$.

models to learn intricate time-dependent probability paths along the flow dimension. Kolloviev et al. (2024) confront precisely this difficulty, introducing informed priors based on Gaussian processes to simplify and accelerate optimization—but at the price of confining their method to univariate forecasting.

We propose FLOWTIME, a simple yet powerful alternative that harnesses FM to learn probabilistic forecasting models that scale to high-dimensional multivariate datasets without any reliance on informed priors. Rather than modeling the entire future window at once as existing FM-based methods do, FLOWTIME decomposes the forecasting problem into a sequence of conditional distributions, modeling the distribution of each future time step conditioned on past observations and covariates through a shared flow that transforms a simple base distribution into the next observation’s distribution. This autoregressive structure is the key to FLOWTIME: it lets the model handle variable forecasting horizons natively, extrapolate reliably beyond the training distribution, and produce well-calibrated uncertainty estimates, all while substantially simplifying the optimization problem.

Our experiments show that FLOWTIME delivers strong performance on both classical dynamical systems and real-world forecasting tasks, providing a practical and effective solution to probabilistic timeseries forecasting.

2 BACKGROUND: FLOW MATCHING

Flow matching (FM) (Lipman et al., 2022) is a generative-modeling paradigm that enables training of continuous normalizing flows (CNFs) (Chen et al., 2018; Grathwohl et al., 2018) at an unprecedented scale. CNFs model data transformations as solutions to ordinary differential equations (ODEs), yielding invertible mappings with tractable likelihoods.

Similar to CNFs, FM aims to learn a time-dependent diffeomorphic map defined on the data space Ω , called a flow $\psi: [0, 1] \times \Omega \rightarrow \Omega$. This flow transforms a sample $\mathcal{X}^0 \sim p$ from a source distribution p into a target sample $\mathcal{X}^1 := \psi(\mathcal{X}^0, 1)$ such that $\mathcal{X}^1 \sim q$ for some target distribution q . The flow is constructed by solving the following initial value problem:

$$d\psi(\mathcal{X}, s) = \mu(\psi(\mathcal{X}, s), s)ds, \quad \psi(\mathcal{X}, 0) = \mathcal{X}, \quad s \in [0, 1] \quad (1)$$

where $\mu: [0, 1] \times \Omega \rightarrow \Omega$ is a vector field defining the velocity of the flow and generating a probability path $(p^s)_{0 \leq s \leq 1}$ where each p^s is a distribution over Ω with $p^0 = p$. The objective is thus to learn a valid vector field μ such that $p^1 = q$.

Unlike CNFs where the vector field is learned via likelihood maximization, requiring solving and differentiating through Eq. (1) during training, FM learns a vector field ν_θ with parameters θ by regressing over vector fields of fixed conditional probability paths. This enables defining a tractable objective function called conditional flow matching (CFM) defined as:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{s \sim \mathcal{U}[0,1], \mathcal{X}^1 \sim q(\mathcal{X}^1), \mathcal{X} \sim p^s(\mathcal{X}|\mathcal{X}^1)} \|\nu_\theta(\mathcal{X}, s) - \mu(\mathcal{X}, s|\mathcal{X}^1)\|^2. \quad (2)$$

Minimizing this objective lets us sample from q by solving Eq. (1) with the trained ν_θ . Because it avoids gradient backpropagation through the ODE solver, FM delivers stable and scalable training for high-dimensional generative tasks such as image and video modeling (Esser et al., 2024; Polyak et al., 2024). A range of probability path designs and conditioning strategies unify multiple transport-based generative models under this single framework (Liu et al., 2022; Albergo et al., 2023; Tong et al., 2023).

3 AUTOREGRESSIVE FLOW MATCHING

Problem Setting Let $\mathbf{y}_\tau \in \mathbb{R}^n$ denote an n -dimensional observation at time τ . Given access to its l past measurements including the current observation, denoted as $\mathbf{Y}_l = \{\mathbf{y}_{\tau-l}, \dots, \mathbf{y}_\tau\}$, our objective is to provide probabilistic forecasts of the next f future values denoted as $\mathbf{Y}_f = \{\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}\}$, potentially conditioned on observed covariates $\mathbf{C} = \{\mathbf{c}_{\tau-l}, \dots, \mathbf{c}_{\tau+f}\}$ where $\mathbf{c}_\tau \in \mathbb{R}^c$. Formally, our goal is to learn to sample from the conditional distribution $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$ using a dataset $\mathcal{D} = \{(\mathbf{Y}_f^i, \mathbf{Y}_l^i, \mathbf{C}^i)\}_{i=1}^m$ with m the number of instances.

We factorize the conditional distribution autoregressively across future time-steps under a Markov assumption of order w :

$$p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C}) = \prod_{t=\tau+1}^{\tau+f} p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \quad (3)$$

where $w \leq l$ is the history size. Our objective is therefore to learn the conditional distribution in Eq. (3). Rather than the maximum likelihood estimation common to autoregressive models, we deliberately learn this conditional distribution with the flow matching framework (Lipman et al., 2022).

This autoregressive factorization delivers four concrete advantages. First, it reduces the learning problem to modeling simple conditional distributions $p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t})$ instead of a complex high-dimensional joint distribution. Second, each time step’s conditional distribution trains within a teacher-forcing framework, enabling fully parallel training. Third, the Markov assumption with window size w gives us direct control over the trade-off between model capacity and computational complexity. Finally, the formulation yields interpretable uncertainty quantification, explicitly modeling how prediction uncertainty propagates through time via the chain of conditional distributions.

3.1 CONSTRUCTING A TARGET PROBABILITY PATH

For all future time steps $t \in [\tau + 1, \tau + f]$, we define a target probability path $(p^s(\mathbf{y}_t))_{0 \leq s \leq 1}$, with marginals $p^0(\mathbf{y}_t) = \mathcal{N}(0, I)$ and $p^1(\mathbf{y}_t) \approx \prod_{t=\tau+1}^{\tau+f} p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \approx p_{\mathcal{D}}$ where $p_{\mathcal{D}}$ is the empirical distribution. This probability path is generated by a flow field $\psi: [0, 1] \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ along s defined via the ordinary differential equation (ODE):

$$d\psi(\mathbf{y}_t, s) = \mu(\psi(\mathbf{y}_t, s), s)ds, \quad \psi(\mathbf{y}_t, 0) = \mathbf{y}_t \quad s \in [0, 1] \quad (4)$$

where $\mu: [0, 1] \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ is a vector field constructing the flow chosen such that the resulting flow satisfy the continuity equation.

To derive a tractable objective, we can model the marginal probability path as a mixture of conditional probability paths:

$$p^s(\mathbf{y}_t) = \int p^s(\mathbf{y}_t|\mathbf{z})p(\mathbf{z})d\mathbf{z} \quad (5)$$

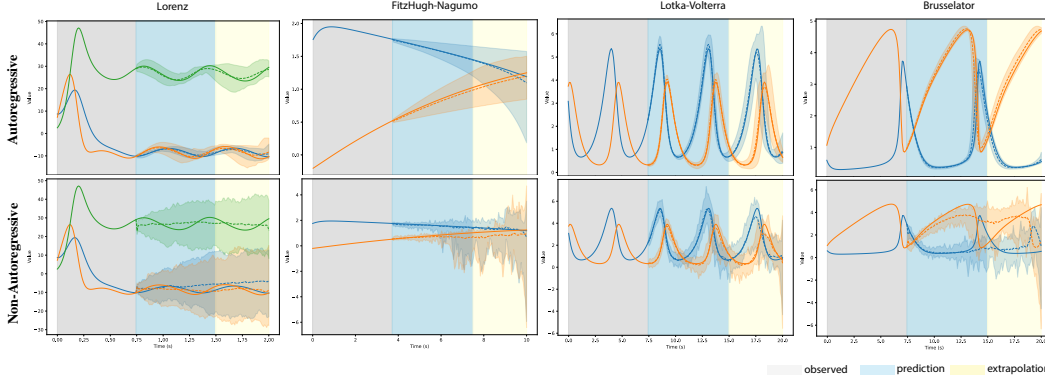


Figure 2: Forecasts from autoregressive flow matching versus standard flow matching on four different dynamical systems. Solid lines show the ground truth; dashed lines show the mean prediction with its 95% confidence interval. Results are shown for both the prediction and the extrapolation regimes, where the autoregressive variant tracks the true dynamics markedly more closely.

where $\mathbf{z} \sim \pi_{0,1}(\mathbf{z})$ is a conditioning random variable sampled from an arbitrary data coupling $\pi_{0,1}$. Theorem 2 in (Lipman et al., 2022) shows that the conditional velocity fields $\mu(\mathbf{y}_t, s|\mathbf{z})$ associated with the conditional probability paths $p^s(\mathbf{y}_t|\mathbf{z})$ are equivalent to their respective marginal velocity field $\mu(\mathbf{y}_t, s)$.

Following (Tong et al., 2023), we choose $\mathbf{z} = \{\mathbf{y}_t^0, \mathbf{y}_t^1\}$ with $p(\mathbf{z} = (\mathbf{y}_t^0, \mathbf{y}_t^1)) = p^0 \times p^1$, where $\mathbf{y}_t^0$ and $\mathbf{y}_t^1$ are samples from p^0 and p^1 , respectively. We define the conditional probability path as:

$$p^s(\mathbf{y}_t|\mathbf{z}) = \mathcal{N}((1-s)\mathbf{y}_t^0 + s\mathbf{y}_t^1, \sigma^2 I) \quad (6)$$

with marginals satisfying $p^0(\mathbf{y}_t|\mathbf{z}) = p^0(\mathbf{y}_t)$ and $p^1(\mathbf{y}_t|\mathbf{z}) = p^1(\mathbf{y}_t)$ for $\sigma^2 \rightarrow 0$. The respective conditional velocity field is then simply $\mu(\mathbf{y}_t, s|\mathbf{z}) = \mathbf{y}_t^1 - \mathbf{y}_t^0$. This choice gives straight probability paths from the source to the target distribution in Euclidean space together with closed-form solutions, which directly translate into efficient training and sampling. Once the velocity field is learned, we sample from our target distribution $p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t})$ simply by solving the ODE in Eq. (4) until $s = 1$.

3.2 TRAINING

Similar to the standard FM setting, we learn this velocity field by regressing it against a neural network ν_θ with parameters θ using the conditional flow matching objective in Eq. 2. To efficiently achieve this in our setting, we define a context vector $\mathbf{h}_t \in \mathbb{R}^h$ that encodes the context window at time t defined as:

$$\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1}) \quad (7)$$

where $\zeta: \mathbb{R}^{w \times n} \times \mathbb{R}^{w \times c} \rightarrow \mathbb{R}^h$ is a neural network with parameters ϕ . We can then define our learning objective as:

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{\mathbf{z} \sim \pi_{0,1}, s \sim \mathcal{U}(0,1), \mathbf{y}_t^s \sim p^s(\mathbf{y}_t|\mathbf{z})} \|\mu(\mathbf{y}_t^s, s|\mathbf{z}) - \nu_\theta(\mathbf{y}_t^s, \mathbf{h}_t, \mathbf{c}_t, s)\|^2 \quad (8)$$

which we optimize by randomly sampling an observation at time point t , its associated context window, and covariates from $\mathcal{D}$, and jointly updating the parameters ϕ and θ to minimize $\mathcal{L}(\theta, \phi)$ via stochastic gradient descent. This is exactly teacher forcing (Williams & Zipser, 1989): we feed ground-truth past observations into the model during training rather than its own past predictions, which enables parallel training across all time-steps and eliminates error accumulation. Algorithm 1 summarizes the training procedure for autoregressive flow matching.

3.3 INFERENCE

After training, we produce probabilistic predictions for new observations $\mathbf{Y}_t$ and corresponding covariates $\mathbf{C}$ across f future timestamps. We follow the sampling procedure in Algorithm 2, generating predictions autoregressively with a rolling-window approach. Repeating this procedure many times yields empirical quantiles that directly characterize the uncertainty of our predictions.

Algorithm 1 Training: Autoregressive Flow Matching.

Require: Dataset $\mathcal{D} = \{(\mathbf{Y}_f^i, \mathbf{Y}_l^i, \mathbf{C}^i)\}_{i=1}^m$, window size w , networks ν_θ, ζ_ϕ

- 1: **while** not converged **do**
- 2: Sample $\mathbf{y}_t^0 \sim \mathcal{N}(0, I)$ and $(\mathbf{y}_t^1, \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \sim \mathcal{D}$
- 3: Set $\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1})$
- 4: Sample $s \sim \mathcal{U}(0, 1)$ and $\mathbf{y}_t^s \sim \mathcal{N}((1-s)\mathbf{y}_t^0 + s\mathbf{y}_t^1, \sigma^2 I)$
- 5: Compute target: $\mu(\mathbf{y}_t^s, s|\mathbf{z}) = \mathbf{y}_t^1 - \mathbf{y}_t^0$
- 6: Compute loss: $\mathcal{L}_t = \|\mu(\mathbf{y}_t^s, s|\mathbf{z}) - \nu_\theta(\mathbf{y}_t^s, \mathbf{h}_t, \mathbf{c}_t, s)\|^2$
- 7: Update θ, ϕ
- 8: **end while**
- 9: **return** ν_θ, ζ_ϕ

Algorithm 2 Sampling: Autoregressive Flow Matching.

Require: Past observations $\mathbf{Y}_l$, covariates $\mathbf{C}$, forecast horizon f , window size w , networks ν_θ, ζ_ϕ

- 1: Initialize $\mathbf{Y}_f \leftarrow \mathbf{Y}_l$
- 2: **for** $t = \tau + 1$ **to** $\tau + f$ **do**
- 3: Set context: $\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1})$, using the last w entries in $\mathbf{Y}_f$
- 4: Sample $\mathbf{y}_t^0 \sim \mathcal{N}(0, I)$
- 5: Solve $\frac{d\psi(\mathbf{y}_t, s)}{ds} = \nu_\theta(\psi(\mathbf{y}_t, s), \mathbf{h}_t, \mathbf{c}_t, s), \quad \psi(\mathbf{y}_t, 0) = \mathbf{y}_t^0, \quad s \in [0, 1]$
- 6: Append $\psi(\mathbf{y}_t, 1)$ to $\mathbf{Y}_f$
- 7: **end for**
- 8: **return** $\mathbf{Y}_f$

4 EXPERIMENTS

In this section, we present our empirical results, comparing FLOWTIME against a comprehensive set of baselines on both real-world datasets and data generated by simulating a range of dynamical systems.

4.1 DYNAMICAL SYSTEMS

Data We evaluated FLOWTIME on the task of forecasting five different stochastic dynamical systems: stochastic versions of the Lorenz system (Lorenz, 1963), FitzHugh–Nagumo model (FitzHugh, 1961), Lotka–Volterra system Lotka (1925), Brusselator (Lefever & Prigogine, 1968), and Van der Pol oscillator (van der Pol, 1926). For each system, we generated trajectories by numerically solving the corresponding stochastic differential equations, using randomly sampled initial conditions and Brownian motion realizations. The details of the dynamical systems and the data generation process are provided in Appendix A.1.

Implementation details For these experiments we set ζ_ϕ as a two-layer bi-directional LSTM with 64 hidden units, and set ν_θ as an MLP with three hidden layers of size 64. We use Fourier positional embeddings to encode the flow step s into a 16-dimensional vector using a fixed set of frequencies and set the context length w to be equal to the prediction length. We trained our models via stochastic gradient descent using an Adam optimizer with a learning rate of 0.003 and a batch size of 128. Training until convergence took ~ 7 minutes on a single A100-SXM4-40GB NVIDIA GPU.

Baselines To isolate the effect of the autoregressive factorization in FLOWTIME, we compare against a non-autoregressive flow-matching baseline that directly models the joint distribution of future observations conditioned on past observations and covariates, i.e. $p^1 = p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$. This comparison is a clean, controlled ablation that cleanly attributes any performance difference to the autoregressive factorization introduced in FLOWTIME. This non-autoregressive formulation closely mirrors the method proposed in (Kollovich et al., 2024). Further details on this baseline are provided in Appendix B.

Evaluation To directly probe the model’s ability to extrapolate beyond its training distribution, we partitioned each trajectory into three segments: (1) an observation window, which serves as the his-

torical input to the model; (2) a prediction window, available only during training as the target; and (3) an extrapolation window, held entirely unseen during training. We measure uncertainty quantification with the continuous rank probability score (CRPS) (Winkler et al., 1996) and prediction accuracy with the normalized root mean square error (NRMSE) (Hyndman & Koehler, 2006).

Table 1: Autoregressive (FLOWTIME) versus non-autoregressive (Non-AR) factorization in our method across five stochastic dynamical systems, reported as CRPS and NRMSE for both the prediction and extrapolation regimes on the test set. Values are the mean and standard deviation across five random seeds; the autoregressive factorization wins on the large majority of settings.

System	Factorization	Prediction		Extrapolation	
		CRPS ↓	NRMSE ↓	CRPS ↓	NRMSE ↓
Lorenz	Non-AR	0.147±0.012	0.242±0.018	0.120 ±0.009	0.301±0.024
	FLOWTIME	0.090 ±0.008	0.017 ±0.003	0.137±0.011	0.048 ±0.006
FitzHugh-Nagumo	Non-AR	0.165±0.014	0.131±0.011	0.214±0.016	0.306±0.022
	FLOWTIME	0.144 ±0.013	0.090 ±0.007	0.146 ±0.012	0.278 ±0.019
Lotka-Volterra	Non-AR	0.130±0.011	0.266±0.019	0.136±0.012	0.502±0.033
	FLOWTIME	0.126 ±0.010	0.189 ±0.015	0.123 ±0.009	0.339 ±0.026
Brusselator	Non-AR	0.217±0.016	0.658±0.042	0.238±0.019	1.046±0.057
	FLOWTIME	0.125 ±0.011	0.031 ±0.004	0.109 ±0.008	0.023 ±0.003
Van der Pol	Non-AR	0.143 ±0.012	0.161±0.014	0.211 ±0.018	0.510±0.031
	FLOWTIME	0.225±0.017	0.051 ±0.005	0.282±0.021	0.101 ±0.009

Results The results in Table 1 establish that the autoregressive factorization in FLOWTIME decisively improves forecasting performance across these stochastic dynamical systems. On the prediction task, the AR approach beats the non-AR baseline in NRMSE on all five systems, with dramatic gains on the Lorenz (0.017 vs 0.242) and Brusselator (0.031 vs 0.658) systems—reductions in error of 93% and 95%, respectively. AR factorization also delivers superior CRPS on four of the five systems, with only the Van der Pol oscillator favoring the non-AR approach on uncertainty quantification. Crucially, these gains carry over to the extrapolation regime, where AR factorization retains its NRMSE advantage on every system, confirming its far stronger ability to generalize beyond the training distribution. The Brusselator case is especially striking: AR factorization cuts the extrapolation NRMSE by 98% (0.023 vs 1.046). Together, these results demonstrate that modeling temporal dependencies through autoregressive factorization enables FLOWTIME to capture the underlying dynamics of complex stochastic systems, yielding more accurate predictions and better uncertainty quantification—most decisively in the demanding extrapolation setting.

4.2 REAL-WORLD DATASETS

Data We evaluated FLOWTIME on the tasks of forecasting multiple univariate and multivariate timeseries with different frequencies (hourly and daily) from GluonTS (Alexandrov et al., 2020). Specifically, we used the following datasets: Electricity (Lai et al., 2018), Solar (Lai et al., 2018), Exchange (Dua et al., 2017), Traffic (Dua et al., 2017), and Wikipedia (Gasthaus et al., 2019). We provide further details in Appendix A.2.

Implementation details We set ζ_ϕ as a three-layer bi-directional LSTM with 64 hidden units. We defined ν_θ via a residual neural network with five residual blocks implemented via 1-D convolutional layers with gated activation functions (Van Den Oord et al., 2016; Kong et al., 2020). The architecture is similar to the network used in (Rasul et al., 2021) but with fewer residual blocks. We used Fourier positional embeddings to encode the flow step s into a 32-dimensional vector using a fixed set of frequencies. We used the same embedding technique to encode time-dependent covariates c . We trained our models via stochastic gradient descent using an Adam optimizer with a learning rate of 0.001 and a batch size of 128. Depending on the dataset, training until convergence took from 40 – 100 minutes on a single A100-SXM4-40GB NVIDIA GPU.

Baselines We benchmark FLOWTIME against a broad and competitive set of established baselines. These span traditional statistical methods—Seasonal Naive (SN), AutoARIMA, and AutoETS (Hyndman et al., 2008)—and deep learning methods including DeepAR (Salinas et al., 2020) and WaveNet (Van Den Oord et al., 2016). We further include strong diffusion baselines TSDiff (Kolloviev et al., 2023), SSSD (Alcaraz & Strodthoff, 2022), and TimeGrad (Rasul et al., 2021), together with the flow matching baseline TSFlow (Kolloviev et al., 2024).

Table 2: Test set CRPS (lower is better) across five real-world datasets. Mean and standard error for FLOWTIME are obtained by re-training and evaluating five times; baseline results are taken from (Kolloviev et al., 2024). Best scores in **bold**, second best underlined. FLOWTIME attains the best score on three of the five datasets.

Method	Electr.	Exchange	Solar	Traffic	Wikipedia
SN	0.060±0.000	0.013±0.000	0.512±0.000	0.221±0.000	0.423±0.000
ARIMA	0.344±0.000	0.013±0.000	0.558±0.003	0.486±0.000	0.654±0.000
ETS	0.056±0.000	0.008±0.000	0.550±0.000	0.492±0.000	0.651±0.000
DeepAR	0.051±0.000	0.013±0.004	0.429±0.015	0.103±0.000	0.215±0.003
WaveNet	0.058±0.008	0.012±0.001	0.360±0.000	0.099±0.000	0.207 ±0.000
CSDI	0.051±0.000	0.013±0.000	0.360±0.000	0.152±0.000	0.318±0.012
SSSD	0.048±0.000	<u>0.010</u> ±0.000	0.354±0.024	0.107±0.002	<u>0.209</u> ±0.000
TSFlow	<u>0.045</u> ±0.001	0.009 ±0.001	<u>0.343</u> ±0.002	0.083 ±0.000	0.227±0.000
FLOWTIME	0.042 ±0.001	0.009 ±0.001	0.284 ±0.002	<u>0.089</u> ±0.000	0.243±0.002

Results Table 2 compares FLOWTIME against the full set of baselines on five real-world timeseries datasets. FLOWTIME sets a new state of the art on three of the five datasets—Electricity, Exchange, and Solar. On Electricity it beats every baseline with a CRPS of 0.042, a 6.7% improvement over the second-best model, TSFlow (0.045). On Exchange it matches the best result, TSFlow, at 0.009, comfortably ahead of the traditional statistical methods and the other deep learning approaches. The gain on Solar is the most emphatic: FLOWTIME reaches a CRPS of 0.284, a 17.2% error reduction over the next-best model, TSFlow (0.343). On Traffic, FLOWTIME places a close second at 0.089, just behind TSFlow (0.083) yet ahead of every other traditional and deep learning baseline. On Wikipedia it remains competitive at 0.243, trailing only WaveNet (0.207) and SSSD (0.209). Taken together, these results show that the autoregressive flow-matching approach in FLOWTIME consistently delivers best-in-class or highly competitive performance across diverse real-world forecasting tasks.

5 DISCUSSION

We introduced FLOWTIME, a probabilistic forecasting model that unites autoregressive modeling with flow matching to produce accurate and well-calibrated predictions for multivariate timeseries data. By factorizing the forecasting problem into a sequence of conditional distributions, FLOWTIME marries the sequential dependencies of temporal data with the expressiveness of flow-based generative modeling. Our experiments across simulated dynamical systems and real-world datasets show that FLOWTIME matches or surpasses state-of-the-art baselines on challenging forecasting benchmarks. The autoregressive decomposition yields clear improvements in uncertainty calibration and distribution coverage over standard flow-matching approaches that model entire future windows at once. FLOWTIME also generalizes robustly to out-of-distribution scenarios—a capability that is essential for real-world deployment, where test conditions routinely diverge from training data.

FLOWTIME also has limitations, which point to concrete opportunities for future work. The sequential nature of autoregressive sampling adds computational overhead at inference relative to fully parallelizable non-autoregressive models, which may constrain its use in real-time forecasting systems. Our current implementation, moreover, applies flow matching directly in the data space, which may be suboptimal for high-dimensional partially observed data. We see latent-space formulations, extensions to irregularly sampled data, and broader applications in scientific modeling and decision-making under uncertainty as promising next steps. FLOWTIME advances probabilistic timeseries forecasting and demonstrates the clear potential of flow-based generative models to improve both accuracy and uncertainty quantification.

REFERENCES

- Michael S Albergo, Nicholas M Boffi, and Eric Vanden-Eijnden. Stochastic interpolants: A unifying framework for flows and diffusions. *arXiv preprint arXiv:2303.08797*, 2023.
- Juan Miguel Lopez Alcaraz and Nils Strodthoff. Diffusion-based time series imputation and forecasting with structured state space models. *arXiv preprint arXiv:2208.09399*, 2022.
- Alexander Alexandrov, Konstantinos Benidis, Michael Bohlke-Schneider, Valentin Flunkert, Jan Gasthaus, Tim Januschowski, Danielle C Maddix, Syama Rangapuram, David Salinas, Jasper Schulz, et al. GluonTS: Probabilistic and neural time series modeling in python. *Journal of Machine Learning Research*, 21(116):1–6, 2020.
- Ricky TQ Chen, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud. Neural ordinary differential equations. *Advances in Neural Information Processing Systems*, 31, 2018.
- Fatoumata Dama and Christine Sinoquet. Time series analysis and modeling to forecast: a survey. In *ArXiv:2104.00164*, 2021.
- Abhyuday Desai, Cynthia Freeman, Zuhui Wang, and Ian Beaver. TimeVAE: A variational auto-encoder for multivariate time series generation. *arXiv preprint arXiv:2111.08095*, 2021.
- Dheeru Dua, Casey Graff, et al. UCI machine learning repository. URL <http://archive.ics.uci.edu/ml>, 7(1):62, 2017.
- Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, et al. Scaling rectified flow transformers for high-resolution image synthesis. In *Forty-first International Conference on Machine Learning*, 2024.
- Richard FitzHugh. Impulses and physiological states in theoretical models of nerve membrane. *Biophysical Journal*, 1(6):445–466, 1961. doi: 10.1016/S0006-3495(61)86902-6.
- Jan Gasthaus, Konstantinos Benidis, Yuyang Wang, Syama Sundar Rangapuram, David Salinas, Valentin Flunkert, and Tim Januschowski. Probabilistic forecasting with spline quantile function RNNs. In *The 22nd International Conference on Artificial Intelligence and Statistics*, pp. 1901–1910. PMLR, 2019.
- Will Grathwohl, Ricky TQ Chen, Jesse Bettencourt, Ilya Sutskever, and David Duvenaud. FFJORD: Free-Form continuous dynamics for scalable reversible generative models. In *International Conference on Learning Representations*, 2018.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33:6840–6851, 2020.
- Yang Hu, Xiao Wang, Lirong Wu, Huatian Zhang, Stan Z. Li, Sheng Wang, and Tianlong Chen. Fm-ts: Flow matching for time series generation. In *ArXiv:2411.07506v1 [cs.LG]*, 2024. URL <http://arxiv.org/abs/2411.07506>.
- Rob Hyndman, Anne B Koehler, J Keith Ord, and Ralph D Snyder. *Forecasting with Exponential Smoothing: The State Space Approach*. Springer Science & Business Media, 2008.
- Rob J Hyndman and Anne B Koehler. Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4):679–688, 2006.
- Maximilian Karl, Maximilian Soelch, Justin Bayer, and Patrick Van der Smagt. Deep variational Bayes filters: Unsupervised learning of state space models from raw data. *arXiv preprint arXiv:1605.06432*, 2016.
- Marcel Kollovich, Abdul Fatir Ansari, Michael Bohlke-Schneider, Jasper Zschiegner, Hao Wang, and Yuyang Bernie Wang. Predict, refine, synthesize: Self-guiding diffusion models for probabilistic time series forecasting. *Advances in Neural Information Processing Systems*, 36:28341–28364, 2023.

432 Marcel Kollovich, Marten Lienen, David Lüdke, Leo Schwinn, and Stephan Günnemann. Flow
433 matching with gaussian process priors for probabilistic time series forecasting. In *The Thirteenth*
434 *International Conference on Learning Representations*, 2024.

435

436 Zhifeng Kong, Wei Ping, Jiaji Huang, Kexin Zhao, and Bryan Catanzaro. DiffWave: A versatile
437 diffusion model for audio synthesis. *arXiv preprint arXiv:2009.09761*, 2020.

438

439 Guokun Lai, Wei-Cheng Chang, Yiming Yang, and Hanxiao Liu. Modeling long- and short-term
440 temporal patterns with deep neural networks. In *The 41st International ACM SIGIR Conference*
441 *on Research & Development in Information Retrieval*, pp. 95–104, 2018.

442 R. Lefever and I. Prigogine. Symmetry breaking instabilities in dissipative systems. II. *The Journal*
443 *of Chemical Physics*, 48(4):1695–1700, 1968. doi: 10.1063/1.1668896.

444

445 Bryan Lim and Stefan Zohren. Time-series forecasting with deep learning: A survey. *Philosophical*
446 *Transactions of the Royal Society A*, 379(2194):20200209, 2021.

447 Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow match-
448 ing for generative modeling. In *The Eleventh International Conference on Learning Representa-*
449 *tions*, 2022.

450

451 Xingchao Liu, Chengyue Gong, et al. Flow straight and fast: Learning to generate and transfer data
452 with rectified flow. In *The Eleventh International Conference on Learning Representations*, 2022.

453

454 Edward Lorenz. Deterministic nonperiodic flow. *Journal of Atmospheric Sciences*, 20(2), 1963.

455

456 Alfred J. Lotka. *Elements of Physical Biology*. Williams & Wilkins, 1925.

457

458 Caspar Meijer and Lydia Y Chen. The rise of diffusion models in time-series forecasting. *arXiv*
preprint arXiv:2401.03006, 2024.

459

460 Adam Polyak, Amit Zohar, Andrew Brown, Andros Tjandra, Animesh Sinha, Ann Lee, Apoorv
461 Vyas, Bowen Shi, Chih-Yao Ma, Ching-Yao Chuang, et al. MovieGen: A cast of media founda-
462 tion models. *arXiv e-prints*, pp. arXiv–2410, 2024.

463

464 Kashif Rasul, Abdul-Saboor Sheikh, Ingmar Schuster, Urs Bergmann, and Roland Vollgraf. Mul-
465 tivariate probabilistic time series forecasting via conditioned normalizing flows. *arXiv preprint*
arXiv:2002.06103, 2020.

466

467 Kashif Rasul, Calvin Seward, Ingmar Schuster, and Roland Vollgraf. Autoregressive denoising dif-
468 fusion models for multivariate probabilistic time series forecasting. In *International Conference*
469 *on Machine Learning*, pp. 8857–8868. PMLR, 2021.

470

471 David Salinas, Michael Bohlke-Schneider, Laurent Callot, Roberto Medico, and Jan Gasthaus.
472 High-dimensional multivariate forecasting with low-rank gaussian copula processes. *Advances*
in Neural Information Processing Systems, 32, 2019.

473

474 David Salinas, Valentin Flunkert, Jan Gasthaus, and Tim Januschowski. DeepAR: Probabilistic
475 forecasting with autoregressive recurrent networks. *International Journal of Forecasting*, 36(3):
476 1181–1191, 2020.

477

478 Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised
479 learning using nonequilibrium thermodynamics. In *International Conference on Machine Learn-*
ing, pp. 2256–2265. PMLR, 2015.

480

481 Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben
482 Poole. Score-based generative modeling through stochastic differential equations. *arXiv preprint*
483 *arXiv:2011.13456*, 2020.

484

485 Ella Tamir, Najwa Laabid, Markus Heinonen, Vikas Garg, and Arno Solin. Conditional flow match-
ing for time series modelling. In *ICML Workshop: Structured Probabilistic Inference and Gener-*
ative Modeling, 2024.

486 Yusuke Tashiro, Jiaming Song, Yang Song, and Stefano Ermon. CSDI: Conditional score-based
487 diffusion models for probabilistic time series imputation. *Advances in Neural Information Pro-*
488 *cessing Systems*, 34:24804–24816, 2021.

489 Alexander Tong, Kilian Fatras, Nikolay Malkin, Guillaume Huguet, Yanlei Zhang, Jarrod Rector-
490 Brooks, Guy Wolf, and Yoshua Bengio. Improving and generalizing flow-based generative models
491 with minibatch optimal transport. *arXiv preprint arXiv:2302.00482*, 2023.

492 Aaron Van Den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves,
493 Nal Kalchbrenner, Andrew Senior, Koray Kavukcuoglu, et al. WaveNet: A generative model for
494 raw audio. *arXiv preprint arXiv:1609.03499*, 12, 2016.

495 B. van der Pol. On ‘relaxation-oscillations’. *The London, Edinburgh, and Dublin Philosophical*
496 *Magazine and Journal of Science*, 2(11):978–992, 1926. doi: 10.1080/14786442608564127.

497 Ronald J Williams and David Zipser. A learning algorithm for continually running fully recurrent
498 neural networks. *Neural Computation*, 1(2):270–280, 1989.

499 Robert L Winkler, Javier Munoz, José L Cervera, José M Bernardo, Gail Blattenberger, Joseph B
500 Kadane, Dennis V Lindley, Allan H Murphy, Robert M Oliver, and David Ríos-Insua. Scoring
501 rules and the evaluation of probabilities. *Test*, 5:1–60, 1996.

502 Jiexia Ye, Weiqi Zhang, Ke Yi, Yongzi Yu, Ziyue Li, Jia Li, and Fugee Tsung. A survey of time
503 series foundation models: Generalizing time series representation with large language model. In
504 *ArXiv:2104.00164*, 2024.

505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539

A DATASETS

A.1 DYNAMICAL SYSTEMS

We generated training and testing data for five classical stochastic dynamical systems, that is, Lorenz, FitzHugh-Nagumo, Lotka-Volterra, Brusselator and Van der Pol, by numerically integrating their corresponding stochastic differential equations (SDEs). Each system has the general form

$$d\mathbf{x}(t) = \mathbf{f}(\mathbf{x}(t), t; \Theta) dt + \sigma d\mathbf{W}(t),$$

where, f is the deterministic drift, Θ are the system parameters, σ is a constant diffusion term, and $\mathbf{W}(t)$ is a standard Brownian motion. We sampled 2400 random initial conditions from a uniform distribution over the specified range (see Table 3) for each system and generated a trajectory for each initial condition by solving the SDE using the Euler-Heun method with fixed step size. We sampled 200 equally spaced time points for each trajectory. Out of the 200 time points in each trajectory, the first 75 were used as observed data, the next 75 for prediction, and the final 50 for extrapolation. Out of the 2400 trajectories generated, we used 2000 for training and 400 for testing.

Table 3: Summary of stochastic dynamical systems. $\mathbf{f}(\mathbf{x})$ denotes the drift function, Θ the system parameters, σ the constant diffusion term, and $p(\mathbf{x}(0))$ the initial condition distribution. Each system was integrated over the given interval $[t_0, t_1]$ with 200 steps using the Euler-Heun method.

System	$f(\mathbf{x})$	Θ	Σ	$p(\mathbf{x}(0))$	$[t_0, t_1]$
Lorenz	$f_1 = \sigma(x_2 - x_1)$	$\sigma = 10$	(1.5, 1.5, 1.5)	$\mathcal{U}([0, 10])$	[0, 2]
	$f_2 = x_1(\rho - x_3) - x_2$	$\rho = 28$			
	$f_3 = x_1 x_2 - \beta x_3$	$\beta = \frac{8}{3}$			
FitzHugh-Nagumo	$f_1 = x_1 - \frac{x_1^3}{3} - x_2 + I$	$a = 0.7$	(1.5, 1.5)	$\mathcal{U}([-2, 2])$	[0, 10]
	$f_2 = \frac{x_1 + a - b x_2}{\tau}$	$b = 0.8$			
		$\tau = 12.5$			
Lotka-Volterra	$f_1 = \alpha x_1 - \beta x_1 x_2$	$\alpha = 1.3$	(1.5, 1.5)	$\mathcal{U}([0, 5])$	[0, 20]
	$f_2 = -\delta x_2 + \gamma x_1 x_2$	$\beta = 0.9$			
		$\gamma = 0.8$			
Brusselator	$f_1 = A + x_1^2 x_2 - (B + 1) x_1$	$A = 1.0$	(1.5, 1.5)	$\mathcal{U}([0, 2])$	[0, 20]
	$f_2 = B x_1 - x_1^2 x_2$	$B = 3.0$			
		$\delta = 1.8$			
Van der Pol	$f_1 = x_2$	$\mu = 0.1$	(1.5, 1.5)	$\mathcal{U}([-2, 2])$	[0, 20]
	$f_2 = \mu(1 - x_1^2) x_2 - x_1$				

A.2 REAL-WORLD DATASETS

We used five different datasets from GluonTS (Alexandrov et al., 2020) to evaluate our model performance. Specifically, we used the versions preprocessed as in (Salinas et al., 2019). A summary of their properties is listed in Table 4.

Table 4: Properties of the datasets used in the experiments: dimension n , domain Ω , frequency, total training timesteps, and prediction length.

Dataset	n	Ω	freq.	timesteps	prediction length
Electricity	370	$\mathbb{R}^+$	hourly	5833	24
Exchange	8	$\mathbb{R}^+$	daily	6071	30
Solar	137	$\mathbb{R}^+$	hourly	7009	24
Traffic	963	(0, 1)	hourly	4001	24
Wikipedia	2000	$\mathbb{N}$	daily	792	30

B NON-AUTOREGRESSIVE FLOW MATCHING BASELINE

Problem Setting We consider the same forecasting setup as Section 3: given past observations $\mathbf{Y}_l$ and covariates $\mathbf{C}$, the goal is to model the conditional distribution $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$. In contrast to the autoregressive factorization, this baseline directly models the joint distribution of the entire future trajectory $\mathbf{Y}_f \in \mathbb{R}^{f \times n}$ with no temporal factorization:

$$p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C}) = p(\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}|\mathbf{Y}_l, \mathbf{C}) \quad (9)$$

This formulation preserves temporal correlations across all future time steps, but at the cost of learning a substantially harder high-dimensional distribution.

Training We construct a probability path $(p^s(\mathbf{Y}_f))^{0 \leq s \leq 1}$ that transports samples from a prior distribution $p^0(\mathbf{Y}_f) = \mathcal{N}(0, \Sigma)$ to the target distribution $p^1(\mathbf{Y}_f) \approx p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$, where Σ is a block-diagonal covariance matrix. The prior can be interpreted as independent Brownian motion processes per dimension, where Brownian motion is defined as a stochastic process $W: [\tau+1, \tau+f] \rightarrow \mathbb{R}^n$.

We define the conditional probability path using a linear interpolation bridge with Brownian noise:

$$p^s(\mathbf{Y}_f|\mathbf{z}) = \mathcal{N}((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1, \sigma^2 s(1-s)\Sigma) \quad (10)$$

where $\mathbf{z} = (\mathbf{Y}_f^0, \mathbf{Y}_f^1)$ with $\mathbf{Y}_f^0 \sim p^0$ and $\mathbf{Y}_f^1 \sim p^1$. The corresponding conditional velocity field becomes:

$$\mu(\mathbf{Y}_f, s|\mathbf{z}) = \mathbf{Y}_f^1 - \mathbf{Y}_f^0 + \frac{\sigma^2(1-2s)}{2}\Sigma^{-1}(\mathbf{Y}_f - ((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1)) \quad (11)$$

When $\sigma^2 \rightarrow 0$, this reduces to the straight path velocity $\mu(\mathbf{Y}_f, s|\mathbf{z}) = \mathbf{Y}_f^1 - \mathbf{Y}_f^0$. We learn a neural velocity field ν_θ that operates on the entire future trajectory. The context encoding remains similar to Eq. (7):

$$\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C}_{\tau-l:\tau+f}) \quad (12)$$

where ζ_ϕ now processes both past and future covariates. The training objective becomes:

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{\mathbf{z} \sim \pi_{0,1}, s \sim \mathcal{U}(0,1), \mathbf{Y}_f^s \sim p^s(\mathbf{Y}_f|\mathbf{z})} \|\mu(\mathbf{Y}_f^s, s|\mathbf{z}) - \nu_\theta(\mathbf{Y}_f^s, \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)\|^2 \quad (13)$$

where ν_θ is implemented as a sequential neural network (e.g., Transformer or RNN) that processes the entire trajectory. Sampling requires solving the trajectory-level ODE:

$$\frac{d\psi(\mathbf{Y}_f, s)}{ds} = \nu_\theta(\psi(\mathbf{Y}_f, s), \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s), \quad \psi(\mathbf{Y}_f, 0) \sim p^0 \quad (14)$$

using numerical solvers. This produces joint samples from $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$ without autoregressive decomposition.

Implementation Details We used this baseline to conduct experiments on the task of forecasting multiple dynamical systems. For this setup we set ν_θ as a 4-layer bi-directional LSTM with 128 hidden units. We kept the same training parameters as in Section 4.1. Training until convergence took ~ 2 minutes on a single A100-SXM4-40GB NVIDIA GPU.

Key Differences from Autoregressive Variant The non-autoregressive approach differs fundamentally from the autoregressive counterpart introduced in FLOWTIME. First, whereas the autoregressive method factorizes the joint distribution via a Markovian decomposition across time steps, the non-autoregressive baseline models the full future trajectory $\mathbf{Y}_f$ as a single high-dimensional random variable, preserving cross-temporal dependencies only by learning a far more complex distribution over $\mathbb{R}^{n \times f}$. Second, the computational profiles diverge sharply: the non-autoregressive version pushes entire trajectories through sequential velocity-field networks, generating all future timepoints in parallel but demanding substantially more memory. Finally, the two contrast in how they handle uncertainty: the autoregressive approach explicitly models how prediction errors accumulate through the chain of conditional distributions, while the non-autoregressive method captures joint uncertainty over all timesteps via trajectory-level sampling but has no explicit mechanism for error-accumulation dynamics. These differences yield complementary trade-offs—the autoregressive variant delivers interpretable uncertainty quantification and efficient window-based computation under a Markovian assumption, while the non-autoregressive baseline retains full temporal correlations at a higher computational cost and with less explicit uncertainty dynamics.

Algorithm 3 Non-Autoregressive Flow Matching: Training

Require: Dataset $\mathcal{D}$, networks ν_θ, ζ_ϕ
1: **while** not converged **do**
2: Sample $\mathbf{Y}_f^0 \sim \mathcal{N}(0, \Sigma)$ and $(\mathbf{Y}_f^1, \mathbf{Y}_l, \mathbf{C}) \sim \mathcal{D}$
3: Set $\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C}_{\tau-l:\tau+f})$
4: Sample $s \sim \mathcal{U}(0, 1)$
5: $\mathbf{Y}_f^s \sim \mathcal{N}((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1, \sigma^2 s(1-s)\Sigma)$
6: Compute target $\mu(\mathbf{Y}_f^s, s|\mathbf{z})$
7: Update θ, ϕ to minimize $\|\mu(\cdot) - \nu_\theta(\mathbf{Y}_f^s, \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)\|_{\Sigma^{-1}}^2$
8: **end while**

Algorithm 4 Non-Autoregressive Flow Matching: Sampling

Require: $\mathbf{Y}_l, \mathbf{C}$, networks ν_θ, ζ_ϕ
1: Encode context: $\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C})$
2: Sample initial trajectory $\mathbf{Y}_f^0 \sim \mathcal{N}(0, \Sigma)$
3: Solve $\frac{d\psi(\mathbf{Y}_f, s)}{ds} = \nu_\theta(\psi(\mathbf{Y}_f, s), \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)$
4: Return $\psi(\mathbf{Y}_f, 1)$

C EVALUATION METRICS

In this section, we describe the metrics used to evaluate the quality of our forecasts. Given an n -dimensional time series $\mathbf{Y}_f = \{\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}\}$ and its predictive distribution or samples from our probabilistic model, we employ two metrics: the Continuous Ranked Probability Score (CRPS) and the Normalized Root Mean Squared Error (NRMSE).

C.1 CONTINUOUS RANKED PROBABILITY SCORE (CRPS)

The CRPS is a proper scoring rule commonly used to assess the quality of probabilistic forecasts. For a univariate random variable X with cumulative distribution function (CDF) F_X and an observed value x , the CRPS is defined as:

$$\text{CRPS}(F_X, x) = \int_{-\infty}^{+\infty} (F_X(y) - \mathbf{1}\{y \geq x\})^2 dy, \quad (15)$$

where $\mathbf{1}\{\cdot\}$ is the indicator function. Intuitively, the CRPS evaluates how well the entire predictive distribution aligns with the observation x . Lower CRPS values indicate better-calibrated and more accurate predictive distributions.

In the context of time series forecasting, we can compute the CRPS for each time step in the forecasting horizon and average the results:

$$\text{CRPS}_{\text{avg}} = \frac{1}{f} \sum_{k=1}^f \text{CRPS}(F_{X_{\tau+k}}, y_{\tau+k}), \quad (16)$$

where $F_{X_{\tau+k}}$ is the predicted CDF (or an empirical CDF from samples) at forecast horizon k , and $y_{\tau+k}$ is the corresponding ground-truth observation.

C.2 NORMALIZED ROOT MEAN SQUARED ERROR (NRMSE)

The Root Mean Squared Error (RMSE) is a standard metric for evaluating the accuracy of point forecasts. For a set of predictions $\{\hat{y}_{\tau+1}, \dots, \hat{y}_{\tau+f}\}$ and corresponding ground-truth values $\{y_{\tau+1}, \dots, y_{\tau+f}\}$, the RMSE is:

$$\text{RMSE} = \sqrt{\frac{1}{f} \sum_{k=1}^f (\hat{y}_{\tau+k} - y_{\tau+k})^2}. \quad (17)$$

To make the RMSE scale-invariant and facilitate comparison across different datasets or time series with different magnitudes, we use the Normalized RMSE (NRMSE). One common normalization is

by the standard deviation of the ground-truth series, σ_y :

$$\text{NRMSE} = \frac{\text{RMSE}}{\sigma_y}, \quad (18)$$

where σ_y is the sample standard deviation of the observations. Alternatively, one could normalize by the range of the data ($\max(y) - \min(y)$) depending on the application. A lower NRMSE indicates more accurate predictions relative to the variability of the time series.